

•



- [Feature Engineering](#)
- [Using PQL in Features](#)
- Reserved Keywords in PQL

On this page

Reserved Keywords in PQL

Was this page helpful? 

Some keywords and symbols cannot be used outside of their original purpose. That means that within a PQL expression, they cannot be used as name of variables, fields or UDFs.

Besides the reserved keywords and symbols, there is a third list of words that cannot be used in PQL expressions due to limitations related to Java.

Reserved keywords

Keyword	Purpose
and	Comparison operator
as	Import aliasing
asc	Order criteria (ascending)
bool	Boolean type
by	Field descriptor (i.e. group by, order by)
desc	Order criteria (descending)
double	Double type
else	Conditional branching
float	Float type
from	Select descriptor (i.e. select from)
group	Grouping operator
if	Conditional branching
import	Import statement for UDFs
in	Variable binding (i.e. let a = 1 in)
int	Integer type
into	Select target (i.e. group by price into ev)
join	Join operator
let	Variable binding (i.e. let a = 1 in)
limit	Limit operator (i.e. limit 1, 10)
long	Long type
on	Join operator
or	Comparison operator
order	Order operator
select	Select operator
Stream	Stream declaration
string	String type
then	Conditional branching
unit	Unit type
where	Select discriminator

Reserved symbols

Symbol	Purpose
.	Field accessor
+	Addition
-	Subtraction
*	Multiplication
/	Division

Symbol	Purpose
%	Division remainder (mod)
^	Exponentiation
<	Less than
<=	Less than or Equal to
=	Assignment
==	Equal
!=	Different
>=	Greater than or Equal to
>	Greater than
{	Block separation
}	Block separation
:	Casting, selection, etc.
;	Statement separation
@	Annotation
??	Coalesce operator

Java keywords

For internal compatibility reasons, the following Java keywords are not allowed:

```

abstract
assert
break
byte
case
catch
char
class
const
continue
default
do
enum
extends
final
finally
for
goto
implements
instanceof
interface
native
new
package
private
protected
public
return
static
strictfp
super
switch
synchronized

```

```
this  
throw  
throws  
transient  
try  
void  
volatile  
while
```

